Architecture Document

Local-First Financial Analysis Application — V1

Status: Draft v1 **Companion to:** `docs/prd.md` **Audience:** Engineers, technical reviewers, future contributors

---

1. Introduction

1.1 Purpose

This document defines the technical architecture for V1 of a local-first macOS financial analysis application. It translates the product requirements in `docs/prd.md` into concrete technology choices, module boundaries, data structures, and runtime behavior. It is the source of truth for "how" the system is built; the PRD is the source of truth for "what" it must do.

1.2 Scope

V1 only. The architecture must accommodate the V2/V3 directions called out in the PRD (multi-company comparison, formula engine, export, plugins) without requiring a rewrite, but no V2+ feature is implemented in V1.

1.3 Key Architectural Drivers

Drawn directly from the PRD and ranked:

1. **Correctness** of financial data — the user must be able to trust every number.
2. **Traceability** — every displayed value must trace back to a specific SEC filing concept.
3. **Local-first** — no cloud backend, full offline operation after ingestion.
4. **Durability** — application crashes must not destroy ingested data.
5. **Modularity** — clean seams for future expansion (peer comparison, formula engine, export).
6. **Maintainability** — a small team / single developer should be able to evolve the codebase.
7. **Performance** — ingestion may be slow; navigation of ingested data must feel instant.

Speed of ingestion is explicitly the *lowest* priority among these.

> **Data acquisition note.** V1 ingests the SEC's pre-extracted XBRL facts via the `data.sec.gov/api/xbrl/companyfacts` JSON API. It does **not** parse 10-K or 10-Q documents — those are filed as HTML / iXBRL / XBRL XML and are out of V1 scope as source material. See §3.5 for the full rationale.

---

2. Architectural Style

The system is a **local-first, modular monolith** packaged as a single desktop application. There is no client/server split, no microservices, and no remote infrastructure.

Internally the codebase is organized as a layered architecture with a separate **pipeline architecture** for the ingestion subsystem:

- **Layered (UI → IPC → Domain Services → Persistence):** for read-side workflows, where the dashboard queries normalized data through repository interfaces.
- **Pipeline (Discover → Download → Parse → Normalize → Persist):** for the ingestion subsystem, where each stage is an isolated, testable unit with explicit inputs and outputs.

This split is deliberate: the read path optimizes for low-latency UI rendering, while the write path optimizes for correctness, observability, and resumability.

---

3. Technology Stack

3.1 Decision Summary

Concern	Choice	Alternative considered
Desktop shell	**Tauri 2.x**	Electron, Swift/AppKit
Backend language	**Rust** (in Tauri's `src-tauri/`)	Go, TypeScript (Node)
Frontend	**React 18 + TypeScript + Vite**	SwiftUI, Svelte
Styling	**Tailwind CSS**	CSS Modules, Stitches
Charting	**Apache ECharts** (via `echarts-for-react`)	Recharts, Chart.js
State / data fetching	**TanStack Query** over Tauri IPC	Redux, Zustand
Local database	**SQLite** via `rusqlite` (bundled)	DuckDB, Postgres
HTTP client	**`reqwest`** with rustls	`ureq`, `hyper`
Async runtime	**`tokio`**	`async-std`
Logging / tracing	**`tracing`** + `tracing-subscriber`	`log` + `env_logger`
Error handling	**`thiserror`** (lib) + **`anyhow`** (app boundary)	manual enums
Migrations	**`refinery`**	hand-rolled, sqlx-migrate
Build / package	**Tauri bundler** → `.dmg`	`cargo-bundle`

3.2 Rationale: Tauri over Electron

- **Bundle size & memory:** Tauri applications are typically ~3–10 MB and use the system WebView (WKWebView on macOS), versus Electron's ~80–150 MB Chromium runtime. For a single-user analytical app, the lighter footprint matters.
- **Security:** Tauri's IPC surface is opt-in per command, with an explicit allowlist. The PRD's "minimize third-party dependencies" and "avoid unnecessary outbound requests" principles map naturally onto Tauri's permission model.
- **Rust backend:** JSON parsing of SEC's `companyfacts` payload, taxonomy normalization, optional XBRL XML fallback parsing, and SQLite work all benefit from Rust's correctness guarantees, error-handling discipline, and zero-cost abstractions. The normalization subsystem in particular is the kind of code where Rust's type system pays for itself.
- **Native macOS look-and-feel:** WKWebView integrates with macOS conventions (text rendering, scrolling, accessibility) more naturally than bundled Chromium.

3.3 Why not native Swift/AppKit?

- Significantly larger surface area for a single-developer effort.

- The financial-domain code (XBRL parsing, normalization) is not language-specific — keeping it in a portable Rust core preserves the option to ship Windows/Linux builds later without rewriting.
- The PRD explicitly lists Tauri as an acceptable approach.

3.4 Rationale: SQLite over DuckDB

DuckDB is more attractive for ad-hoc analytical queries, but for V1:

- The dataset is small (one company × ~80 quarters × ~200 facts ≈ 16 K rows; even 50 companies stays well under 1 M rows).
- SQLite has more mature tooling, ACID guarantees the PRD requires for durability, and a more stable on-disk format for long-term local storage.
- Tauri + `rusqlite` has a well-trodden integration path.
- DuckDB can be added later as a read-only analytical accelerator without changing the canonical store.

3.5 Rationale: how V1 acquires financial data from SEC EDGAR

This is the most important data-flow decision in V1. It is worth being precise about what is and is not on the wire.

10-K and 10-Q documents themselves are not in JSON. A filing is submitted to EDGAR as a bundle of HTML / inline XBRL (iXBRL) / XBRL XML and exhibits. There is no public API endpoint that returns "the 10-K, as JSON." Anyone parsing a 10-K is parsing HTML or XML.

What V1 actually consumes is a different artifact. Since the SEC XBRL mandate (large filers ~2009, all U.S.-listed filers ~2011), every numeric line item on the financial statements in a 10-K, 10-Q, or financial-statement 8-K must be tagged with a structured XBRL concept (e.g., `us-gaap:Revenues`, `us-gaap:Assets`, `us-gaap:NetIncomeLoss`). The SEC ingests those tagged values into a structured database and exposes a public read-side at:

```
https://data.sec.gov/api/xbrl/companyfacts/CIK{cik10}.json
```

This JSON is **not the 10-K**. It is the SEC's own pre-extracted view of every XBRL fact the company has ever tagged in a 10-K / 10-Q / 8-K, aggregated into a single JSON document keyed by taxonomy and concept. Each fact in the response carries:

Field	Meaning
`val`	The numeric value, in the declared unit
`unit` (key)	`USD`, `shares`, `USD/shares`, etc.
`start`, `end`	Period covered (or `end` only, for instant facts)
`fy`	Fiscal year
`fp`	Fiscal period (`FY`, `Q1`, `Q2`, `Q3`, `Q4`)
`form`	Source form type (`10-K`, `10-Q`, `10-K/A`, `8-K`, …)
`accn`	Accession number of the source filing

The `accn` is what gives V1 its source-filing traceability for free (FR-060): every normalized value can be linked back to the exact filing it came from without us touching the filing itself.

What V1 gets from `companyfacts`: every tagged numeric line item on the income statement, balance sheet, and cash-flow statement, across every reporting period the company has filed, for any U.S.-listed company that complies with the XBRL mandate. This is exactly the data the V1 PRD requires (FR-020 / FR-021 / FR-022).

What V1 does *not* get from `companyfacts`:

- MD&A narrative

- Footnote prose
- Risk factors and other Reg S-K narrative sections
- Exhibits
- Anything inside the 10-K that is not XBRL-tagged

None of this is in V1 scope. The PRD explicitly defers narrative analysis (earnings call analysis, AI-generated insights) to V3.

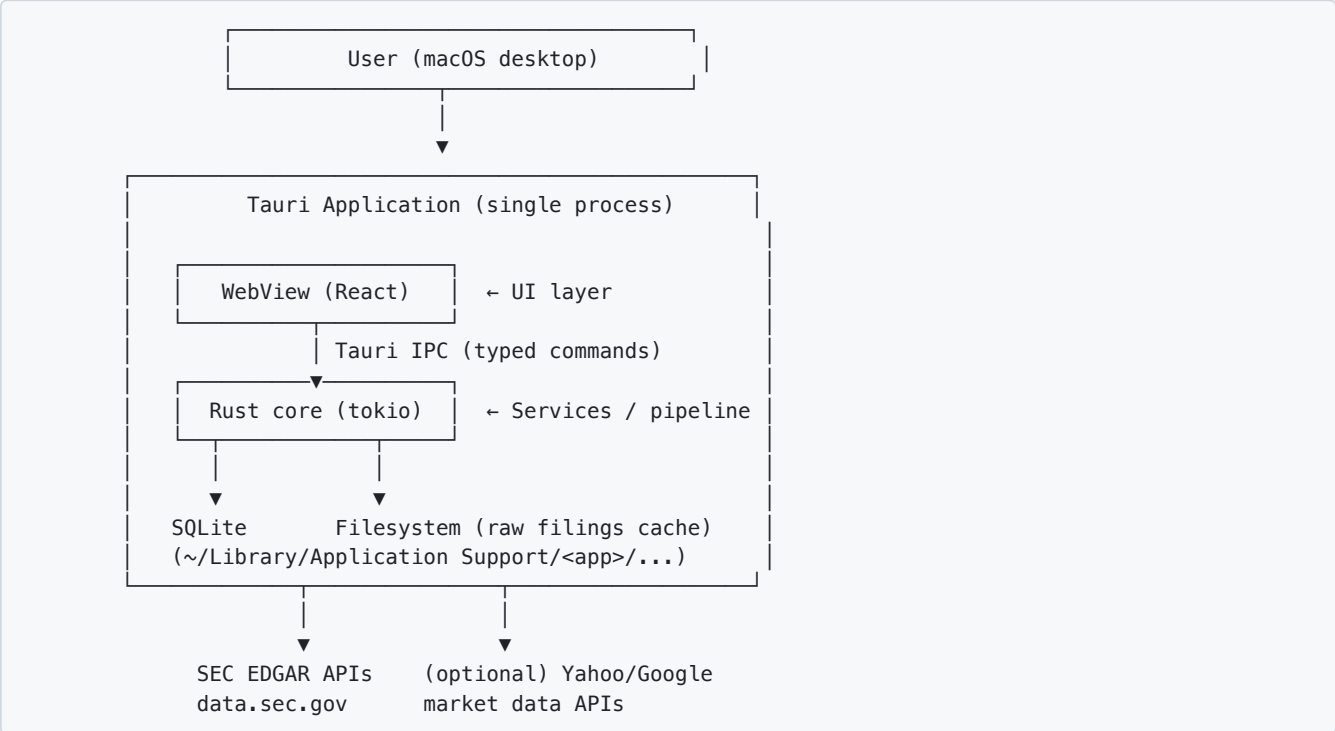
Consequence: V1 never parses a 10-K document. The only document the ingestion pipeline ever reads in the normal path is `companyfacts.json` (one per company) plus the `submissions.json` filing index. A secondary, fallback path can fetch the raw **XBRL instance document** (XML) inside a specific filing for the rare case where `companyfacts` is missing a value the company actually reported. That fallback parses XBRL XML, **not 10-K HTML**. HTML / iXBRL parsing of 10-Ks is explicitly out of V1 scope and is deferred to V3.

The fallback is implemented behind the same `FactSource` trait as the JSON path, so it can be enabled per-concept-per-filing without disturbing the rest of the pipeline.

---

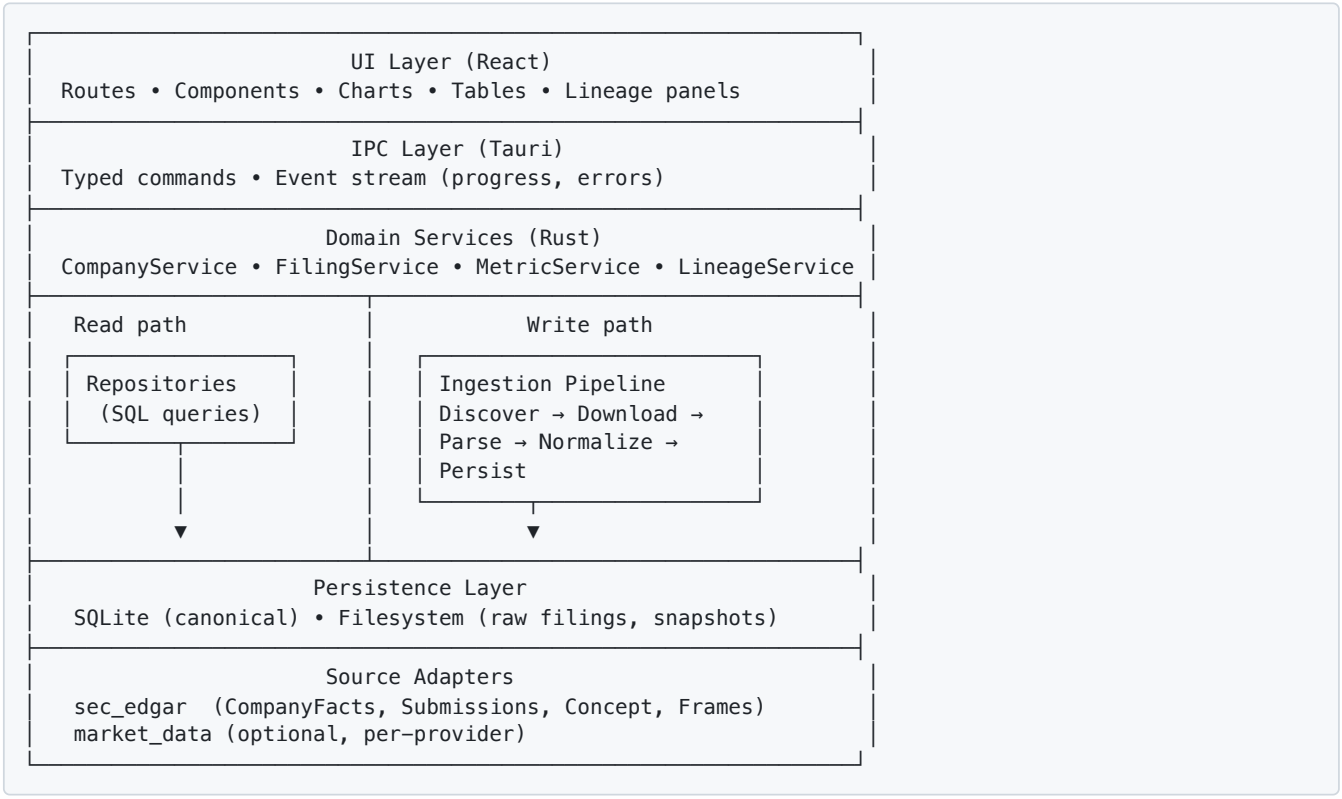
4. High-Level Architecture

4.1 System Context



There is exactly one outbound dependency that is required for ingestion: the SEC EDGAR APIs. Market-data APIs are optional and the application degrades gracefully without them.

4.2 Layer Diagram



The strict downward dependency rule applies: each layer depends only on layers below it. The UI never reaches into the database directly; the ingestion pipeline never calls into the UI; source adapters never know about SQL.

---

5. Module Structure

5.1 Repository layout

```
econ_project/
├── docs/                          # PRD, architecture, future ADRs
├── src/                           # React app
│   ├── api/                       #   Typed Tauri IPC client
│   ├── features/
│   │   ├── home/                  #   Saved companies, add ticker
│   │   ├── company/               #   Dashboard, charts, tables
│   │   ├── lineage/               #   Drill-down to filing source
│   │   └── ingestion/             #   Progress UI
│   ├── components/                #   Generic UI primitives
│   ├── charts/                    #   ECharts wrappers
│   ├── state/                     #   TanStack Query hooks
│   └── styles/
├── src-tauri/                     # Rust core
│   ├── src/
│   │   ├── main.rs
│   │   ├── ipc/                   #   #[tauri::command] entry points
│   │   ├── domain/                #   Service layer (use-cases)
│   │   ├── ingestion/             #   Pipeline stages
│   │   │   ├── discover.rs
│   │   │   ├── download.rs
│   │   │   ├── parse.rs
│   │   │   ├── normalize.rs
│   │   │   └── persist.rs
│   │   ├── normalization/         #   Concept maps, unit/period rules
│   │   │   ├── concept_map.rs
│   │   │   ├── periods.rs
│   │   │   ├── units.rs
│   │   │   └── signs.rs
│   │   ├── sources/
│   │   │   ├── sec_edgar/         #   API clients + raw XBRL fallback
│   │   │   └── market_data/       #   Optional, behind feature flag
│   │   ├── persistence/
│   │   │   ├── db.rs              #   Connection pool, migrations
│   │   │   ├── repositories/      #   One module per aggregate
│   │   │   └── schema/            #   .sql migrations
│   │   ├── derived/               #   Derived-metric formulas
│   │   ├── lineage/               #   Source-tracking helpers
│   │   ├── errors.rs
│   │   └── telemetry.rs           #   tracing setup (local logs only)
│   └── Cargo.toml
└── README.md
```

5.2 Crate / module boundaries

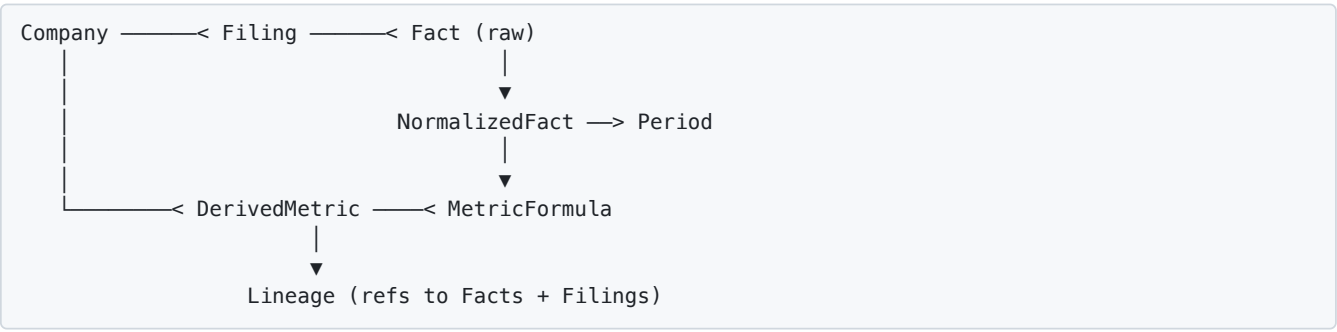
The Rust code is one binary crate but organized so that each top-level module under `src-tauri/src/` could be extracted into its own crate later without churn. In particular:

- `normalization/` depends on no I/O, only on domain types — it is pure logic and should be heavily unit-tested.
- `sources/sec_edgar/` depends on `reqwest` and the raw EDGAR JSON shape, but exposes only typed `RawFact` / `RawFiling` records to the rest of the system.
- `persistence/repositories/` is the only module allowed to write SQL.

---

6. Data Architecture

6.1 Conceptual model



6.2 Canonical metric model

The product's correctness hinges on a stable internal vocabulary. V1 ships a hard-coded **canonical metric catalog** — a small enum of well-defined financial concepts with documented semantics:

Canonical metric	Statement	Aggregation	Sign convention
`revenue`	Income	Period flow	Positive
`cost_of_revenue`	Income	Period flow	Positive
`gross_profit`	Income	Period flow	Positive
`operating_income`	Income	Period flow	Signed
`net_income`	Income	Period flow	Signed
`eps_basic`	Income	Period flow	Signed
`eps_diluted`	Income	Period flow	Signed
`shares_outstanding_basic`	Income	Period instant	Positive
`shares_outstanding_diluted`	Income	Period instant	Positive
`cash_and_equivalents`	Balance	Instant	Positive
`total_debt`	Balance	Instant	Positive
`total_assets`	Balance	Instant	Positive
`total_liabilities`	Balance	Instant	Positive
`total_equity`	Balance	Instant	Signed
`cash_from_operations`	Cash flow	Period flow	Signed
`capital_expenditures`	Cash flow	Period flow	**Stored positive** (sign-normalized)
`depreciation_amortization`	Cash flow	Period flow	Positive

Each entry records its statement, whether it is a flow (period) or stock (instant), and the canonical sign convention used at storage time. UI display rules can then invert signs consistently for presentation (e.g., showing CapEx as a negative cash outflow on a cash-flow waterfall).

The catalog is a Rust enum so missing values are caught at compile time. New metrics are added by extending the enum, the concept map, and (where applicable) a derived formula.

6.3 SQLite schema (V1)

```
-- Companies
CREATE TABLE company (
  cik             TEXT PRIMARY KEY,           -- 10-digit zero-padded
  ticker          TEXT NOT NULL,
  name            TEXT NOT NULL,
  exchange        TEXT,
  sic             TEXT,
  fiscal_year_end TEXT,                       -- MMDD
  added_at        TEXT NOT NULL,              -- ISO8601
  last_refreshed  TEXT
);
CREATE UNIQUE INDEX idx_company_ticker ON company(ticker);

-- Filings (one row per accession number per company)
CREATE TABLE filing (
  accession_no    TEXT PRIMARY KEY,           -- e.g. "0000320193-24-000123"
  cik             TEXT NOT NULL REFERENCES company(cik),
  form_type       TEXT NOT NULL,              -- 10-K, 10-Q, 10-K/A, ...
  filed_at        TEXT NOT NULL,              -- ISO date
  period_of_report TEXT,                      -- ISO date
  is_amendment    INTEGER NOT NULL DEFAULT 0,
  amends          TEXT,                       -- accession_no this amends
  source_url      TEXT,
  raw_path        TEXT                        -- local file path if cached
);
CREATE INDEX idx_filing_cik_filed ON filing(cik, filed_at DESC);

-- Periods (canonicalized fiscal periods)
CREATE TABLE period (
  id              INTEGER PRIMARY KEY,
  cik             TEXT NOT NULL REFERENCES company(cik),
  fiscal_year     INTEGER NOT NULL,
  fiscal_quarter  INTEGER,                    -- NULL for annual
  start_date      TEXT NOT NULL,
  end_date        TEXT NOT NULL,
  kind            TEXT NOT NULL,              -- 'annual' | 'quarterly'
  UNIQUE (cik, fiscal_year, fiscal_quarter, kind)
);

-- Raw facts (1:1 with what we got from EDGAR / XBRL)
CREATE TABLE raw_fact (
  id              INTEGER PRIMARY KEY,
  cik             TEXT NOT NULL,
  accession_no    TEXT NOT NULL REFERENCES filing(accession_no),
  taxonomy        TEXT NOT NULL,              -- 'us-gaap', 'ifrs-full', ...
  concept         TEXT NOT NULL,              -- e.g. 'Revenues'
  unit            TEXT NOT NULL,              -- 'USD', 'shares', 'USD/shares'
  scale           INTEGER NOT NULL DEFAULT 0, -- power-of-ten applied
  value_numeric   REAL NOT NULL,              -- already scaled to base unit
  period_start    TEXT,
  period_end      TEXT NOT NULL,
  is_instant      INTEGER NOT NULL,
  fy              INTEGER,
  fp              TEXT,                       -- 'FY','Q1','Q2','Q3','Q4'
  ingested_at     TEXT NOT NULL
);
CREATE INDEX idx_raw_cik_concept ON raw_fact(cik, taxonomy, concept);
CREATE INDEX idx_raw_filing ON raw_fact(accession_no);
```

```
-- Canonical / normalized facts
CREATE TABLE normalized_fact (
  id              INTEGER PRIMARY KEY,
  cik             TEXT NOT NULL,
  metric          TEXT NOT NULL,              -- canonical metric name
  period_id       INTEGER NOT NULL REFERENCES period(id),
  value           REAL NOT NULL,              -- in base unit (USD, shares, ...)
  unit            TEXT NOT NULL,
  source_fact_id  INTEGER NOT NULL REFERENCES raw_fact(id),
  source_kind     TEXT NOT NULL,              -- 'xbrl_api' | 'xbrl_xml' (V1 does not parse 10-K HTML)
  confidence      REAL NOT NULL DEFAULT 1.0,
  superseded_by   INTEGER REFERENCES normalized_fact(id),
  ingested_at     TEXT NOT NULL,
  UNIQUE (cik, metric, period_id, source_fact_id)
);
CREATE INDEX idx_norm_cik_metric_period ON normalized_fact(cik, metric, period_id);

-- Derived metrics (computed from normalized facts)
CREATE TABLE derived_metric (
  id              INTEGER PRIMARY KEY,
  cik             TEXT NOT NULL,
  formula_id      TEXT NOT NULL,              -- 'fcf_v1', 'gross_margin_v1', ...
  period_id       INTEGER NOT NULL REFERENCES period(id),
  value           REAL,
  is_complete     INTEGER NOT NULL,           -- 0 if any input missing
  computed_at     TEXT NOT NULL,
  UNIQUE (cik, formula_id, period_id)
);

-- Ingestion diagnostics (observable to the user)
CREATE TABLE ingestion_event (
  id              INTEGER PRIMARY KEY,
  cik             TEXT,
  accession_no    TEXT,
  stage           TEXT NOT NULL,              -- discover|download|parse|normalize|persist
  level           TEXT NOT NULL,              -- info|warn|error
  message         TEXT NOT NULL,
  detail_json     TEXT,
  occurred_at     TEXT NOT NULL
);
CREATE INDEX idx_ing_cik_time ON ingestion_event(cik, occurred_at DESC);
```

Notes:

- `raw_fact` and `normalized_fact` are kept as **separate tables** rather than overwriting raw data. This is what makes traceability cheap: a normalized row points at exactly one raw row, which points at exactly one filing.
- `superseded_by` lets restated values point at their replacement without losing history. The UI selects `WHERE superseded_by IS NULL` for current values; the lineage panel can show the supersession chain.
- The schema is migration-managed; every change ships as a numbered SQL file, not a hand edit.

6.4 Filesystem layout

```
~/Library/Application Support/com.<vendor>.econ-project/
├── data.sqlite                              # canonical DB
├── data.sqlite-wal                          # WAL (journal_mode=WAL)
├── filings/
│   └── <cik>/
│       ├── companyfacts.json                # SEC's pre-extracted XBRL facts, one per company
│       ├── submissions.json                 # SEC filing index, one per company
│       └── <accession_no>/                  # only created on XBRL-XML fallback
│           ├── primary_doc.xml              # raw XBRL instance document (NOT the 10-K HTML)
│           └── metadata.json
├── logs/
│   └── econ-project.<date>.log
└── config.json                              # user prefs
```

The two top-level files under each `<cik>/` directory (`companyfacts.json`, `submissions.json`) are the only artifacts V1 fetches in the normal path — both are per-company, not per-filing. The per-`<accession_no>/` subdirectory is created only when the fallback path needs to retrieve a specific filing's XBRL instance document; it is empty for most companies.

Raw payloads are kept on disk (not in SQLite blobs) so they can be inspected, backed up, or re-processed without touching the database. The `raw_path` column in `filing` points to the relevant directory or file.

---

7. SEC Integration

7.1 Endpoints used

Purpose	Endpoint	Notes
Ticker → CIK lookup	`https://www.sec.gov/files/company_tickers.json`	Cached locally; refresh weekly
Filing index	`https://data.sec.gov/submissions/CIK{cik10}.json`	Per-company list of filings (form type, accession, dates). Used to know what filings exist; does **not** contain financial values.
All facts (primary)	`https://data.sec.gov/api/xbrl/companyfacts/CIK{cik10}.json`	**SEC's pre-extracted XBRL facts**, aggregated across all of a company's 10-K / 10-Q / 8-K filings. **Not the filings themselves.** One JSON per company. This is V1's main data source.
Single concept (fallback)	`https://data.sec.gov/api/xbrl/companyconcept/CIK{cik10}/{taxonomy}/{tag}.json`	Same data sliced by concept. Used for targeted re-fetch when `companyfacts` has a gap for a known concept.
Raw filing index	`https://www.sec.gov/cgi-bin/browse-edgar?...`	Used only to locate the path of a specific filing's XBRL instance document when the fallback path is required.
Raw XBRL document	`https://www.sec.gov/Archives/edgar/data/{cik}/{accession-stripped}/{primary_doc}`	The **XBRL instance document (XML)** inside a specific filing.

Purpose	Endpoint	Notes
		Last-resort parse. V1 does **not** download or parse the 10-K HTML / iXBRL document.

7.2 Compliance

- **User-Agent:** the SEC requires a descriptive `User-Agent: AppName/Version contact@example.com`. The string is configurable in `config.json` and surfaced in onboarding so the user understands what is being sent.
- **Rate limit:** SEC publishes a 10 requests/second ceiling. The client enforces a token-bucket limiter at 5 req/s by default with jittered backoff on 429/5xx.
- **No mass crawling:** the app fetches per-company on user action, never speculatively.

7.3 Caching and refresh

- `company_tickers.json` cached for 7 days.
- `submissions/CIK{cik}.json` cached for 24 hours, with `If-Modified-Since` revalidation.
- `companyfacts/CIK{cik}.json` cached forever; refreshed on user-initiated refresh, with `ETag` / `Last-Modified` revalidation.
- Raw filings are immutable once fetched — accession numbers never get reused.

7.4 Failure modes

Condition	Response
No network	UI surfaces "offline"; ingestion buttons disabled; navigation of cached data unaffected
429 rate limit	Exponential backoff up to 60s, then surface a user-visible warning
5xx	Same as 429 with a different message
Schema change in EDGAR JSON	Fail closed for that fact; record an `ingestion_event` with the offending payload; do not overwrite previously-good data
Missing concept for a known metric	Mark the period as having a gap; the UI shows an explicit "—"

---

8. Normalization Engine

The PRD elevates normalization to a *core architectural requirement*. The engine is implemented as a pure-functions module with the following responsibilities:

8.1 Concept mapping

A concept map associates external XBRL concepts with canonical metrics:

```
// Sketch
ConceptMap {
    revenue: [
        ("us-gaap", "Revenues"),
        ("us-gaap", "RevenueFromContractWithCustomerExcludingAssessedTax"),
        ("us-gaap", "SalesRevenueNet"),
        ("us-gaap", "SalesRevenueGoodsNet"),
        ("us-gaap", "SalesRevenueServicesNet"),
    ],
    capital_expenditures: [
        ("us-gaap", "PaymentsToAcquirePropertyPlantAndEquipment"),
        ("us-gaap", "PaymentsToAcquireProductiveAssets"),
    ],
    // ...
}
```

When multiple candidates produce values for the same `(metric, period)`, a **resolution rule** selects one and records the others as alternates with reduced confidence:

1. Prefer the concept marked as the canonical primary in the catalog.
2. Prefer values from the most recent non-amended filing covering the period.
3. Prefer values from amendments only when the amendment explicitly supersedes the original (`amends` chain).

The decision and its inputs are written to `ingestion_event` so the user can audit it later.

8.2 Period reconciliation

The most error-prone area. The engine:

- Aligns fiscal periods to the company's `fiscal_year_end`. A company with FYE 09-30 (like Apple) has its FY2024 covering Oct 2023 – Sep 2024.
- Distinguishes **instant** facts (balance-sheet items at a point in time) from **duration** facts (income/cash-flow items over a period).
- Handles **YTD vs. quarterly** reporting: many filings report Q3 as the 9-month total. The engine derives Q3-only by subtracting Q1 and Q2 when needed, and records the derivation in lineage.
- Detects **fiscal-year-end changes** (rare but real, e.g., Microsoft pre-1986) and explicitly warns rather than silently re-aligning.

8.3 Unit normalization

Each `raw_fact` records its declared unit and scale. Normalization converts to base units:

- Currency facts → base currency (USD for V1; multi-currency support is V2).
- Share counts → absolute share counts.
- Per-share ratios → kept as ratios with explicit `USD/shares` unit.

If a fact arrives in a non-USD currency, V1 stores it in `raw_fact` but does not promote it to `normalized_fact`; an `ingestion_event` records the skip.

8.4 Sign normalization

Cash-flow concepts in particular use mixed sign conventions across companies (CapEx may be reported as positive payments or negative cash flow). The catalog defines the storage sign convention; the normalizer applies it deterministically and records both the original sign and the applied transform in lineage.

8.5 Restatement handling

When a 10-K/A or 10-Q/A is ingested:

1. New `filing` row marked `is_amendment = 1`, `amends = <original accession>`.
2. New `raw_fact` and `normalized_fact` rows inserted.
3. Previous `normalized_fact` rows for the same `(cik, metric, period)` get `superseded_by` pointing at the new row.
4. Read queries default to `WHERE superseded_by IS NULL`.
5. The lineage panel shows the supersession chain on demand.

8.6 Conflict surfacing

The PRD says: "*The application must never silently discard normalization conflicts or ambiguities.*"

Every place the engine has to make a choice writes an `ingestion_event`. These events power a "Data quality" tab in the company dashboard: missing periods, conflicting concepts, derived-from-YTD subtractions, unit mismatches, etc.

---

9. Ingestion Pipeline

9.1 Stages

```
Discover → Download → Parse → Normalize → Persist
```

Each stage takes a typed input and produces a typed output; each is independently testable with golden fixtures.

Stage	Input	Output	Side effects
Discover	Ticker	CIK + filing index from `submissions.json`	None
Download	CIK	`companyfacts.json` (primary path); per-accession XBRL instance XML (fallback only)	Filesystem
Parse	`companyfacts.json` (and any fallback XBRL XML)	`RawFact` records	None
Normalize	RawFacts + ConceptMap	NormalizedFacts + Diagnostics	None
Persist	NormalizedFacts + RawFacts + Diagnostics	Persisted state	DB

Normal-path traffic. For a typical ingestion, V1 makes exactly two HTTP requests against `data.sec.gov`: one for `submissions/CIK{cik10}.json` and one for `xbrl/companyfacts/CIK{cik10}.json`. Per-filing requests are made *only* on the fallback path (a specific concept missing for a specific filing in `companyfacts`). 10-K HTML / iXBRL documents are never fetched.

9.2 Concurrency model

- Stages run as `tokio` tasks within a per-ingestion job.
- Network I/O (Discover, Download) is rate-limited globally via a shared token bucket so multiple ingestion jobs cannot exceed SEC rate limits.
- DB writes happen on a single writer task that owns the SQLite write connection; readers use a separate pool. This avoids `SQLITE_BUSY` while WAL is in use.

- Progress is published over a Tauri event channel (`ingestion://progress/<job_id>`) for the UI to render progress bars.

9.3 Idempotency

- Ingestion is keyed by `accession_no`. Re-running ingestion for an already-persisted filing is a no-op.
- A partially-completed job can be resumed: the persist stage records a checkpoint per filing, and Discover skips filings already at the latest stage.

9.4 Failure semantics

Per the PRD, partial ingestion is acceptable as long as it is visible. The pipeline:

- **Continues** when a single filing fails to parse — other filings still produce data.
- **Halts** the job when the database itself is unhealthy (disk full, corruption signal).
- **Records every failure** as an `ingestion_event` row so the user can see what was skipped.

---

10. Derived Metric Engine

V1 ships a small fixed catalog of formulas. Each formula is a Rust function that:

1. Declares the canonical metrics it consumes.
2. Returns `Option<f64>` plus a `lineage` record listing the input facts.

```
// Sketch
fn fcf_v1(inputs: &MetricInputs) -> DerivedResult {
    let ni  = inputs.get(Metric::NetIncome)?;
    let dep = inputs.get(Metric::DepreciationAmortization)?;
    let cx  = inputs.get(Metric::CapitalExpenditures)?;
    DerivedResult::ok(ni + dep - cx, lineage![ni, dep, cx])
}
```

If any input is missing, the result is `is_complete = 0` with no value, and the UI displays a gap rather than a fabricated number. This satisfies FR-030, FR-031, FR-032 and the PRD principle of "explicit gaps, no fabricated estimates."

The engine is built so V2's user-defined formulas can plug in via a registry — but V1 only registers compile-time functions.

---

11. UI Architecture

11.1 Routes

Route	Purpose
`/`	Home: saved companies, add ticker, refresh status
`/c/:ticker`	Company dashboard
`/c/:ticker/statement/:kind`	Income / Balance / Cash flow tables
`/c/:ticker/metric/:metric`	Drill-down: per-metric history + lineage
`/c/:ticker/diagnostics`	Ingestion diagnostics & data-quality view

11.2 Component composition

- **Dashboard** = `<SummaryWidgets/>` + `<ChartGrid/>` + `<StatementsTable/>`.
- **SummaryWidgets** displays revenue, net income, cash, debt, FCF as latest TTM with a sparkline.
- **ChartGrid** exposes Annual / Quarterly toggle (FR-051), default 10y, configurable to 20y.
- **StatementsTable** is a virtualized table (TanStack Table) with row drill-down → `/c/:ticker/metric/:metric`.
- **Lineage panel** is a side drawer that shows: filing accession, form type, filing date, XBRL concept, raw value, scale, sign transform, supersession chain (FR-060/061/062).

11.3 State and data flow

- All data is fetched via Tauri IPC commands wrapped as TanStack Query hooks.
- The query cache is the de-facto in-memory state; there is no Redux.
- Mutations (add company, refresh) invalidate relevant queries and trigger re-fetch.
- Long-running jobs (ingestion) emit progress events; the UI subscribes via Tauri event listeners.

11.4 Design system

- Density: ~13 px base font; tabular numerals for all financial values.
- Color: a constrained palette (neutrals + one accent) with explicit positive/negative colors for variances. No gradients, no shadows beyond elevation-1.
- Typography: a single sans-serif (system stack: SF Pro on macOS) plus a mono stack for raw values in the lineage panel.
- Loading states: skeleton placeholders for tables; explicit "missing data" markers (FR-105, error visibility).

---

12. Concurrency, Performance, and Caching

12.1 SQLite tuning

```
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA temp_store  = MEMORY;
PRAGMA mmap_size   = 268435456; -- 256 MB
PRAGMA foreign_keys = ON;
```

WAL is essential so the UI can read while ingestion writes.

12.2 Connection model

- One **write** connection, owned by the persist worker.
- A **read pool** of 4 connections shared across IPC handlers.

12.3 Query patterns

- The dashboard's hot path is `SELECT metric, period, value FROM normalized_fact WHERE cik = ? AND superseded_by IS NULL ORDER BY period_id` — covered by `idx_norm_cik_metric_period`.
- All frequent reads are bounded by `cik`; an index on `(cik, ...)` covers them.

12.4 Caching

- Filing metadata: in DB, `last_refreshed` per company.
- Derived metrics: persisted in `derived_metric`; recomputed only when an input changes.
- UI: TanStack Query with a 5-minute stale time for IPC results inside a session.

---

13. Observability

13.1 Logging

- `tracing` with a JSON layer to a rotating file under `~/Library/.../logs/`.
- A console layer when launched with `RUST_LOG=debug`.
- Log levels: `info` for stage transitions, `warn` for fallbacks and conflicts, `error` for skipped data.

13.2 In-app diagnostics

Every interesting decision the system makes during ingestion writes a row to `ingestion_event`. The **Diagnostics** tab in the company dashboard renders this table with filters. This is what makes the PRD's "transparency" requirement concrete.

13.3 No telemetry

Nothing is sent off the device. The app does not ping a vendor URL, does not check for updates, and does not phone home. This is enforced by the Tauri allowlist (only `data.sec.gov` and the optional market-data domain are permitted for HTTP).

---

14. Error Handling

14.1 Layered errors

- **Source layer:** `SourceError` with variants for HTTP status, parse failures, schema violations.
- **Pipeline layer:** `IngestionError` wraps `SourceError` and adds stage context.
- **Service layer:** `AppError` is the top-level enum exposed across IPC; it carries a stable `code` string for the UI to switch on, plus a localized message.

14.2 IPC error contract

Every Tauri command returns `Result<T, AppError>`. The frontend's typed client maps `AppError.code` to user-visible messaging; the underlying message and detail are surfaced in a "Show details" affordance for advanced users.

14.3 Partial success

A common case: ingestion succeeds for 18 of 20 filings. The IPC response is `Ok(IngestionSummary { succeeded, skipped: [...] })` and the UI shows a non-blocking notice with a link to Diagnostics.

---

15. Security and Privacy

Concern	Mitigation
Outbound traffic	Tauri allowlist + a `reqwest` client built from a fixed allowlist of hosts
Telemetry	None — zero analytics, zero remote config
Local data integrity	SQLite WAL + `synchronous = NORMAL`; periodic `PRAGMA integrity_check` on startup
Filesystem permissions	Data stored under `~/Library/Application Support/<bundle>/`, app-sandbox-friendly
Code signing	Tauri signed and notarized for distribution (deferred to release; documented but not blocking)
Dependencies	Minimal set; `cargo audit` in CI; npm audit for the React side
Secrets	None stored; SEC has no auth; the User-Agent contact email is user-supplied

---

16. Build, Packaging, and Distribution

- Local development: `pnpm tauri dev`.
- Production build: `pnpm tauri build` produces a `.dmg` and `.app`.
- Unit tests: `cargo test` for Rust, `vitest` for the React side.
- Integration tests: golden-fixture tests that ingest a checked-in copy of an EDGAR `companyfacts` JSON for a sample company (e.g., Apple) and assert the resulting normalized rows.
- CI (GitHub Actions, post-V1): macOS runner, lint + test + bundle.
- Distribution: V1 is a developer-distributed `.dmg`. App Store distribution is V2+.

---

17. Future Extensibility

The architecture has explicit seams for the V2/V3 directions in the PRD:

Future feature	Seam
Multi-company comparison	Repository queries are already keyed by `cik`; UI introduces a new layer that joins multiple companies. No schema change.
User-defined formulas	The derived-metric engine already accepts a registry; V2 adds a parser + sandboxed evaluator.
Peer benchmarking	A `peer_group` table joins companies; the metric service gains a `for_peers` API.
Export	A new `export/` module reads from `normalized_fact` and `derived_metric` and writes CSV/Excel/PDF. The data is unchanged.
Plugin SDK	A stable Rust trait (`MetricProvider`, `SourceAdapter`) is already what the in-tree modules implement. Plugins ship as dynamic libraries loaded behind a permission prompt.
Local LLM	The lineage and normalization layers already produce structured records that can be fed as context.

The boundary that matters most is keeping `raw_fact` separate from `normalized_fact`. Every future feature will benefit from the audit trail this provides.

---

18. Risks and Open Questions

Risk	Likelihood	Mitigation
EDGAR companyfacts API changes its schema	Low	Fail closed per fact; raw XBRL fallback; integration tests on golden fixtures
Concept-map coverage is incomplete for some industries (banks, insurers, REITs)	Medium	V1 ships a vetted catalog for non-financials; insurers/banks flagged as "limited coverage" until V2
Companies with non-USD reporting currency	Medium	V1 stores raw, skips normalization, surfaces a clear notice
Restatement chains spanning >1 amendment	Low	`superseded_by` is recursive; UI walks the chain
User runs ingestion for many companies in parallel and hits SEC rate limits	Medium	Global token bucket; queue rather than reject
SQLite corruption from disk-full	Low	WAL + integrity check on startup; copy-on-write filesystem (APFS) helps
Ticker→CIK lookup ambiguity (multiple share classes)	Medium	Show disambiguation UI when the lookup returns >1 candidate

Open questions to resolve before / during implementation

1. **Reporting currency support** — does V1 quietly skip non-USD filers, or refuse to add them? Recommendation: refuse with a clear message; defer to V2.
2. **Quarterly derivation policy** — when a Q3-only value isn't directly reported, should the system derive it (Q3 = 9-month YTD − Q2 YTD) or leave a gap? Recommendation: derive, with explicit lineage. The PRD prefers transparency over gaps when the math is unambiguous.
3. **Market cap source** — Yahoo Finance's API is unstable; should V1 ship without market cap rather than depend on it? Recommendation: yes, ship without; add an opt-in market-data adapter when a reliable free source is identified.
4. **Code signing & notarization timing** — required before any external distribution. Plan to set up Apple Developer ID before the first user-facing build.

---

19. Appendices

A. Sample SEC URLs (for reference)

- Apple ticker→CIK: `https://www.sec.gov/cgi-bin/browse-edgar?action=getcompany&CIK=AAPL&type=10-K&dateb=&owner=include&count=40`
- Apple submissions: `https://data.sec.gov/submissions/CIK0000320193.json`
- Apple company facts: `https://data.sec.gov/api/xbrl/companyfacts/CIK0000320193.json`
- Apple revenues concept: `https://data.sec.gov/api/xbrl/companyconcept/CIK0000320193/us-gaap/Revenues.json`

B. Glossary

- **CIK** — Central Index Key, SEC's unique 10-digit company identifier.
- **XBRL** — eXtensible Business Reporting Language; the XML-based standard used in SEC structured filings.
- **Taxonomy** — a published vocabulary of XBRL concepts (e.g., `us-gaap`, `ifrs-full`).
- **Concept** — a single tagged datapoint within a taxonomy (e.g., `us-gaap:Revenues`).

- **Accession number** — SEC's unique identifier for a single filing submission.
- **Form 10-K / 10-Q** — annual / quarterly report.
- **/A suffix** — amendment (e.g., 10-K/A).
- **Instant vs. duration** — XBRL distinction between point-in-time facts (balance sheet) and over-period facts (income / cash flow).
- **Fiscal period** — a company-defined accounting period; may not align with the calendar year.

C. Document conventions

- Code blocks marked `// Sketch` are illustrative, not final API.
- Schema DDL in §6.3 is the canonical baseline for migration `0001_initial.sql`.
- Any deviation from the catalog or DDL during implementation must be captured in a follow-up ADR under `docs/adr/`.